

SIMATS ENGINEERING

CO-4_ASSESSMENT TOOL 1

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA0613

NAME:MURALI KRISHNAN N

REG NO: 192524090

COURSE OUTCOME COVERED

CO 4 : Solve optimization problems using dynamic programming and greedy strategies.(BL4, BL5)

Coding Challenge Task

Q25. Palindrome Partitioning

1. Problem Statement

Given a string, partition it into substrings such that each substring is a palindrome. The objective is to minimize the number of cuts required to achieve such a partition. Efficient identification of palindromic substrings is necessary, and Dynamic Programming is used to determine the minimum cuts needed.

Input Format:

- s = input string

Sample Input / Output:

- $s = \text{"aab"}$
- Min Cuts = 1 (Partition: "aa" | "b")

2. Approach and Documentation

The solution uses a two-stage Dynamic Programming approach:

- Stage 1 — Palindrome Table: A 2D boolean table $\text{is_palindrome}[i][j]$ is built bottom-up, where $\text{is_palindrome}[i][j]$ is True if the substring $s[i..j]$ is a palindrome. A substring is a palindrome if its end characters match and the inner substring is also a palindrome (or has length ≤ 2).
- Stage 2 — Minimum Cuts: A 1D array $\text{cuts}[i]$ stores the minimum cuts needed to partition $s[0..i]$ into palindromes. If $s[0..i]$ is itself a palindrome,

$\text{cuts}[i] = 0$. Otherwise, $\text{cuts}[i] = \min(\text{cuts}[j] + 1)$ for every $j < i$ where $s[j+1..i]$ is a palindrome.

- The final answer is $\text{cuts}[n-1]$, the minimum cuts for the whole string.

Functions:

- `min_cut_palindrome_partition(s)` — returns the minimum number of cuts.
- `get_partition(s)` — reconstructs and returns one valid minimum-cut partition, used to illustrate the result.

How to Run:

- Save the source file as `palindrome_partition.py`
- Run using: `python3 palindrome_partition.py`
- The script executes the sample input and several additional test cases automatically.

3. Source Code (Screenshot)

```
1  """
2  Q25. Palindrome Partitioning - Minimum Cuts
3  Dynamic Programming Approach
4
5  Given a string s, partition it into substrings such that every substring
6  is a palindrome. Determine the minimum number of cuts required.
7  """
8
9  def min_cut_palindrome_partition(s: str) -> int:
10     n = len(s)
11     if n == 0:
12         return 0
13
14     # is_palindrome[i][j] = True if s[i..j] is a palindrome
15     is_palindrome = [[False] * n for _ in range(n)]
16     for i in range(n):
17         is_palindrome[i][i] = True
18
19     for length in range(2, n + 1):
20         for i in range(n - length + 1):
21             j = i + length - 1
22             if s[i] == s[j]:
23                 if length == 2 or is_palindrome[i + 1][j - 1]:
24                     is_palindrome[i][j] = True
25
26     # cuts[i] = minimum cuts needed for s[0..i]
27     cuts = [0] * n
28     for i in range(n):
29         if is_palindrome[0][i]:
30             cuts[i] = 0
31         else:
32             cuts[i] = float('inf')
33             for j in range(i):
34                 if is_palindrome[j + 1][i] and cuts[j] + 1 < cuts[i]:
35                     cuts[i] = cuts[j] + 1
36
37     return cuts[n - 1]
38
39
40 def get_partition(s: str):
41     """Reconstructs one valid minimum-cut palindrome partition for display."""
42     n = len(s)
43     is_palindrome = [[False] * n for _ in range(n)]
44     for i in range(n):
45         is_palindrome[i][i] = True
46     for length in range(2, n + 1):
47         for i in range(n - length + 1):
48             j = i + length - 1
49             if s[i] == s[j] and (length == 2 or is_palindrome[i + 1][j - 1]):
50                 is_palindrome[i][j] = True
51
52     cuts = [0] * n
53     split_at = [-1] * n
54     for i in range(n):
55         if is_palindrome[0][i]:
56             cuts[i] = 0
57         else:
58             cuts[i] = float('inf')
59             for j in range(i):
60                 if is_palindrome[j + 1][i] and cuts[j] + 1 < cuts[i]:
61                     cuts[i] = cuts[j] + 1
62                     split_at[i] = j
63
64     parts = []
65     i = n - 1
66     while i >= 0:
67         j = split_at[i]
68         parts.append(s[j + 1:i + 1])
69         i = j
70     parts.reverse()
71     return parts
72
73
74 if __name__ == "__main__":
75     test_cases = ["aab", "a", "aa", "ab", "racecarxyzabba", "abcde", ""]
76     for s in test_cases:
77         result = min_cut_palindrome_partition(s)
78         partition = get_partition(s) if s else []
79         print(f"Input: s = '{s}'")
80         print(f"Min Cuts = {result}")
81         print(f"Sample Partition: {partition}")
82         print("-" * 40)
```

4. Output (Screenshot)

```
Terminal Output

$ python3 palindrome_partition.py
Input: s = 'aab'
Min Cuts = 1
Sample Partition: ['aa', 'b']
-----
Input: s = 'a'
Min Cuts = 0
Sample Partition: ['a']
-----
Input: s = 'aa'
Min Cuts = 0
Sample Partition: ['aa']
-----
Input: s = 'ab'
Min Cuts = 1
Sample Partition: ['a', 'b']
-----
Input: s = 'racecarxyzabba'
Min Cuts = 4
Sample Partition: ['racecar', 'x', 'y', 'z', 'abba']
-----
Input: s = 'abcde'
Min Cuts = 4
Sample Partition: ['a', 'b', 'c', 'd', 'e']
-----
Input: s = ''
Min Cuts = 0
Sample Partition: []
-----
```

For the required sample input `s = "aab"`, the program correctly outputs Min Cuts = 1, matching the expected output.

5. Test Case Summary

#	Input (s)	Expected Min Cuts	Sample Partition	Result
1	"aab"	1	['aa', 'b']	Pass
2	"a"	0	['a']	Pass
3	"aa"	0	['aa']	Pass
4	"ab"	1	['a', 'b']	Pass

#	Input (s)	Expected Min Cuts	Sample Partition	Result
5	"racecarxyzabba"	4	['racecar','x','y','z','abba']	Pass
6	"abcde"	4	['a','b','c','d','e']	Pass
7	"" (empty)	0	[]	Pass

6. Analysis

Time Complexity:

- Building the palindrome table: $O(n^2)$, since every substring (i, j) pair is checked once in $O(1)$ using previously computed results.
- Computing the cuts array: $O(n^2)$ in the worst case, since for each position i, up to i previous positions j are checked.
- Overall Time Complexity: $O(n^2)$.

Space Complexity:

- $O(n^2)$ for the palindrome table, plus $O(n)$ for the cuts array $\rightarrow O(n^2)$ overall.

Why Dynamic Programming:

- A brute-force approach would try every possible partition, which is exponential ($O(2^n)$). Dynamic Programming avoids recomputation by reusing previously solved subproblems (overlapping subproblems and optimal substructure), reducing the complexity to $O(n^2)$, which is efficient enough for competitive-programming constraints.

Edge Cases Considered:

- Empty string $\rightarrow$ 0 cuts.
- Single character $\rightarrow$ already a palindrome, 0 cuts.
- Entire string already a palindrome $\rightarrow$ 0 cuts.
- No palindromic substrings longer than 1 $\rightarrow$ $n-1$ cuts (worst case).